



Programi për Shkenca Kompjuterike dhe Inxhinierise

NETWORK INTRUSION DETECTION

Shkalla Bachelor

Delvinë Bekteshi, Flandra Bytyqi

Qershor / 2026
Prishtinë



Programi për Shkenca Kompjuterike dhe Inxhinierise

Dokumentim i Projektit
Viti akademik 2025 – 2026

Delvinë Bekteshi, Flandra Bytyqi

NETWORK INTRUSION DETECTION

Mentori: **Greta Ahma, PhD (c)**

Qershor / 2026

1. Hyrje

Me rritjen e përdorimit të rrjeteve kompjuterike dhe shërbimeve online, siguria kibernetike është bërë një nga sfidat më të rëndësishme të sistemeve moderne informatike. Sulmet ndaj rrjeteve mund të shkaktojnë humbje të të dhënave, ndërprerje të shërbimeve, komprometim të privatësisë dhe dëme të konsiderueshme financiare e operative. Për këtë arsye, organizatat mbështeten gjithnjë e më shumë në sisteme inteligjente që janë në gjendje të identifikojnë sjellje të dyshimta ose malicioze në trafikun e rrjetit.

Një Intrusion Detection System (IDS) ka për qëllim të monitorojë trafikun e rrjetit dhe të dallojë aktivitetin normal nga ai potencialisht sulmues. Qasjet tradicionale të bazuara në rregulla dhe nënshkrime janë efektive për sulmet e njohura, por shpesh kanë vështirësi në zbulimin e modeleve të reja ose varianteve të panjohura të sulmeve. Për këtë arsye, machine learning është bërë një alternativë shumë e rëndësishme në ndërtimin e IDS-ve më fleksibile dhe më adaptive.

Qëllimi i këtij projekti është ndërtimi dhe krahasimi i disa modeleve të machine learning për detektimin e intruzioneve në rrjet, duke përdorur dataset-in NSL-KDD. Projekti fokusohet në klasifikimin binar të trafikut në dy klasa kryesore, normal dhe attack, duke implementuar modele të ndryshme të mësimit të mbikëqyrur dhe duke i vlerësuar ato me metrika standarde si accuracy, precision, recall, F1-score dhe confusion matrix.

Përveç klasifikimit të mbikëqyrur, projekti përfshin edhe një analizë të pambikëqyrur me **KMeans clustering**, me qëllim të eksplorimit të strukturës së brendshme të të dhënave. Për vizualizimin e kësaj pjese është përdorur edhe **Principal Component Analysis (PCA)**, e cila ndihmon në reduktimin dimensional dhe në paraqitjen grafike të cluster-ave në një hapësirë dy-dimensionale.

Ky projekt synon jo vetëm të demonstrojë se si ndërtohet një pipeline i plotë i machine learning për intrusion detection, por edhe të tregojë se si qasjet e ndryshme — lineare, tree-based, distance-based, neural dhe clustering — sillen mbi të njëjtin problem dhe të njëjtin dataset. Në këtë mënyrë, projekti ofron një krahasim praktik dhe metodologjik të teknikave kryesore që mund të përdoren në ndërtimin e një sistemi inteligjent për zbulimin e sulmeve në rrjet.

2. Përshkrimi i Datasetit

Dataset-i i përdorur në këtë projekt është NSL-KDD, i cili është një version i përmirësuar i dataset-it të njohur KDD'99 për intrusion detection. Ky dataset u krijua për të adresuar disa nga problemet më të rëndësishme të KDD'99, sidomos praninë e rekordeve të tepërta dhe vlerësimin e padrejtë të modeleve për shkak të shpërndarjes jo të përshtatshme të mostrave. Për këtë arsye, NSL-KDD vazhdon të përdoret si benchmark akademik për krahasimin e algoritmeve të machine learning në cybersecurity.

Sipas përshkrimeve të dataset-it, NSL-KDD përmban rreth 125,973 rekorde në pjesën e trajnimit dhe rreth 22,544 rekorde në pjesën e testimit. Çdo rekord përmban 41 veçori që përshkruajnë sjelljen e një lidhjeje rrjeti, ndërsa kolonat shitesë përfshijnë etiketën e klasës dhe difficulty level. Kjo e bën dataset-in mjaft të pasur për të testuar si klasifikimin ashtu edhe analizat e pambikëqyrura.

Veçoritë e NSL-KDD zakonisht grupohen në katër kategori kryesore:

- **Basic/Intrinsic features**, që përshkruajnë vetë lidhjen, si p.sh. kohëzgjatja dhe lloji i protokollit.
- **Content features**, që lidhen me përmbajtjen e trafikut dhe me disa indikatorë të sjelljes së përdoruesit.
- **Time-based traffic features**, që matin sjelljen e rrjetit në një dritare kohore të shkurtër.
- **Host-based traffic features**, që përshkruajnë statistika të lidhura me hostin në periudha më të gjata.

Disa nga veçoritë më të rëndësishme që përdoren në praktikë përfshijnë *duration*, *protocol_type*, *service*, *flag*, *src_bytes*, *dst_bytes*, *logged_in*, *count*, *srv_count* dhe shumë veçori të tjera që pasqyrojnë sjelljen e lidhjes dhe aktivitetin e rrjetit. Një pjesë e këtyre veçorive janë **kategorike**, si *protocol_type*, *service* dhe *flag*, ndërsa shumica tjetër janë **numerike**, gjë që e bën preprocessing-un një hap thelbësor para trajnimit të modeleve.

Etiketat e sulmeve në NSL-KDD përfshijnë lloje të ndryshme sjelljesh malicioze dhe zakonisht ndahen në katër kategori kryesore:

- **DoS (Denial of Service)**
- **Probe**
- **R2L (Remote to Local)**
- **U2R (User to Root)**

Në këtë projekt, problemi nuk është trajtuar si multi-class classification, por si **binary classification**. Kjo do të thotë se të gjitha llojet e sulmeve janë bashkuar në një klasë të vetme **attack**, ndërsa trafiku legjitim është lënë si klasë **normal**. Në mënyrë konkrete, etiketa është transformuar në:

- **0 = normal**
- **1 = attack**

Kjo qasje e thjeshton analizën dhe e bën më të lehtë interpretimin e rezultateve, sidomos në një projekt kursi ku fokusi është krahasimi i modeleve dhe jo dallimi i hollësishëm mes llojeve individuale të sulmeve.

Përzgjedhja e NSL-KDD si dataset për këtë projekt është e arsyeshme, sepse ai ofron një kombinim të mirë mes kompleksitetit të mjaftueshëm, strukturës së njohur dhe përdorimit të gjerë në literaturë. Megjithatë, duhet theksuar se edhe ky dataset ka kufizime, pasi nuk përfaqëson në mënyrë të plotë trafikun modern të rrjeteve reale dhe nuk përfshin të gjitha llojet bashkëkohore të sulmeve. Prandaj, rezultatet e marra duhen interpretuar si një analizë metodologjike dhe eksperimentale, më shumë sesa si matje absolute e performancës së një sistemi IDS në mjedis real.

3. Metodologjia

Metodologjia e ndjekur në këtë projekt është ndërtuar si një pipeline e plotë e machine learning për detektimin e intruzioneve, duke përfshirë ngarkimin dhe transformimin e të dhënave, preprocessing-un, trajnimin e modeleve të klasifikimit, optimizimin e hiperparametrave, evaluimin e performancës dhe analizën clustering me vizualizim. Kjo qasje lejon një krahasim të drejtë dhe të strukturuar mes metodave të ndryshme.

Përgatitja e të dhënave

Në hapin e parë, dataset-i u lexua nga file-i përkatës dhe kolonat u emërtuan sipas formatit standard të NSL-KDD. Pas kësaj, etiketa e klasës u transformua në format binar, ku trafiku “normal” u kodua si 0 dhe të gjitha llojet e sulmeve si 1. Ky hap ishte i domosdoshëm për ta kthyer problemin në një binary classification task, i cili është më i thjeshtë për t’u analizuar dhe përputhet mirë me qëllimin kryesor të projektit.

Më pas u ndanë veçoritë hyrëse XX nga etiketa yy. Kjo ndarje krijoi bazën për trajtimin e të dhënave me algoritme të ndryshme të machine learning, duke siguruar që modelet të trajtoheshin vetëm mbi inputet dhe jo mbi klasën target.

Preprocessing

Për shkak se NSL-KDD përmban si veçori numerike ashtu edhe kategorike, preprocessing ishte një hap thelbësor i pipeline-it. Veçoritë kategorike si `protocol_type`, `service` dhe `flag` u transformuan me **one-hot encoding**, sepse modelet e machine learning zakonisht nuk mund t’i interpretojnë drejtpërdrejt tekstet ose kategoritë simbolike. One-hot encoding krijon kolona binare për secilën kategori dhe shmang futjen e një rendi artificial midis vlerave kategorike.

Pas kodimit, veçoritë numerike u standardizuan me **StandardScaler**. Ky hap është i rëndësishëm sidomos për modele si Logistic Regression, KNN dhe Neural Network, sepse këto modele ndikohen nga madhësitë relative të inputeve dhe nga distancat në hapësirën e të dhënave. Standardizimi i sjell veçoritë në një shkallë më të krahasueshme dhe ndihmon në stabilitetin dhe performancën e modeleve.

Pas përfundimit të preprocessing-ut, të dhënat u ndanë në train set dhe test set. Train set u përdor për trajnim dhe tuning, ndërsa test set u ruajt për vlerësimin përfundimtar të modeleve mbi të dhëna të paparë më parë. Kjo qasje ndihmon në shmangien e optimizimit të tepruar ndaj të dhënave të trajnimit dhe jep një pasqyrë më realiste të aftësisë generalizuese të modelit.

```
Starting Data Preprocessing Pipeline

Preprocessing Completed Successfully!
X_train shape: (100778, 119)
X_test shape: (25195, 119)

Target label distribution (Training Set):
label
0      53921
1      46857
Name: count, dtype: int64
```

Figura 1. Output i pipeline-it të preprocessing: dimensionet e X_train/X_test dhe shpërndarja e label-it në train set.

Modelit e klasifikimit

Në projekt u përfshinë katër klasifikues të ndryshëm, secili që përfaqëson një qasje të ndryshme të machine learning:

- **Logistic Regression**, si model linear bazë për binary classification
- **Decision Tree**, si model tree-based që mund të kapë rregulla dhe marrëdhënie jo-lineare
- **K-Nearest Neighbors (KNN)**, si model që bazohet në afërsinë midis mostrave
- **Neural Network**, si model më fleksibël dhe jo-linear

Kjo përzgjedhje u bë me qëllim që të krahasoheshin qasje me natyrë të ndryshme mbi të njëjtin problem. Logistic Regression shërben si baseline linear, Decision Tree ofron interpretueshmëri dhe kapacitet për ndarje jo-lineare, KNN ofron një qasje intuitive të bazuar në distancë, ndërsa Neural Network përfaqëson një model me kapacitet më të lartë për të mësuar struktura komplekse në të dhëna.

Hyperparameter tuning

Për të rritur performancën e modeleve dhe për të shmangur përdorimin e parametrave default, u aplikua **GridSearchCV** për tre modele klasike: Logistic Regression, Decision Tree dhe KNN. GridSearchCV provon kombinime të ndryshme hiperparametrash dhe

zgjedh konfigurimin që jep rezultatin më të mirë sipas një metrike të caktuar në cross-validation. Kjo e bën procesin e tuning-ut sistematik dhe të riprodhueshëm.

Në këtë projekt:

- Për Logistic Regression u testuan vlera të ndryshme të C , me $penalty='l2'$ dhe $solver='lbfgs'$
- Për Decision Tree u testuan max_depth , $min_samples_split$ dhe $min_samples_leaf$.
- Për KNN u testuan $n_neighbors$, $weights$ dhe p .

Për Neural Network, tuning-u nuk u bë me grid search të plotë, por me **dy eksperimente manuale arkitekturash**. U ndërtuan dy rrjete nervore me struktura të ndryshme për të krahasuar nëse një arkitekturë më e thellë dhe më komplekse sjell vërtet përmirësim ndaj një arkitekture më të thjeshtë. Kjo ishte një zgjedhje praktike, sepse tuning-u i thellë i rrjeteve nervore kërkon më shumë kohë dhe resurse llogaritëse.

Evaluimi i performancës

Për secilin model të klasifikimit u përdorën metrika standarde:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-score**
- **Confusion Matrix**

Zgjedhja e këtyre metrikave është e rëndësishme në intrusion detection, sepse accuracy e vetme nuk jep gjithmonë informacion të mjaftueshëm për cilësinë reale të modelit. Në veçanti, recall është kritik për të matur sa mirë modeli kap sulmet, ndërsa precision ndihmon për të kuptuar sa alarmet e ngritura janë reale. F1-score, si balancë mes precision dhe recall, është veçanërisht i dobishëm kur synohet një kompromis i mirë mes ndjeshmërisë dhe saktësisë.

Confusion matrix u përdor për të parë në mënyrë më të detajuar numrin e rasteve të klasifikuara saktë dhe gabim në secilën klasë. Kjo ndihmon në interpretimin e false positives dhe false negatives, të cilat janë shumë të rëndësishme në kontekstin e sigurisë së rrjetit.

Clustering dhe PCA

Përveç analizës së mbikëqyrur, projekti përfshiu edhe një pjesë të pambikëqyrur me **KMeans clustering**. Kjo u bë për të analizuar nëse të dhënat formojnë grupe natyrale që afrohen me klasat reale normal/attack, pa përdorur labels gjatë trajnimit.

U testuan disa vlera të ndryshme të k , si p.sh. 2, 3 dhe 5. Pas trajnimit të KMeans, cluster-at e krijuar u krahasuan me etiketat reale të dataset-it përmes metrikave **Adjusted Rand Index (ARI)** dhe **Normalized Mutual Information (NMI)**. Këto dy metrika matin shkallën e përputhjes mes clustering-ut dhe klasifikimit real.

Për vizualizim u përdor **PCA (Principal Component Analysis)** me dy komponentë. PCA redukton dimensionalitetin e të dhënave duke ruajtur sa më shumë informacion nga varianca origjinale. Sipas dokumentacionit të scikit-learn, atributi *explained_variance_ratio_* tregon përqindjen e variancës së shpjeguar nga secili komponent i zgjedhur, dhe kjo ndihmon për të kuptuar sa informacion ruhet në projeksionin 2D.

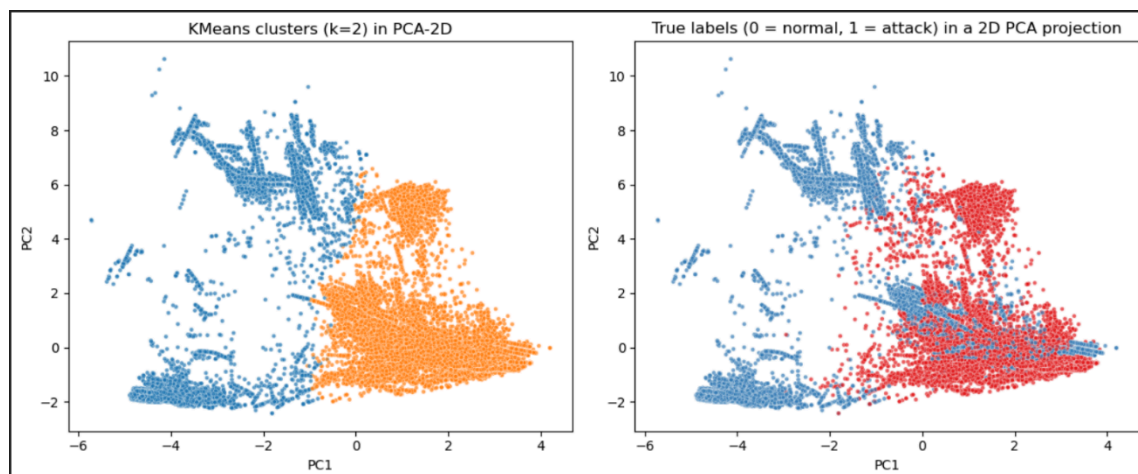


Figura 2. PCA-based vizualizimi i KMeans clusters ($k = 2$) dhe true class labels.

4. Rezultatet

Rezultatet e projektit u ndanë në dy pjesë kryesore: rezultatet e klasifikimit të mbikëqyrur dhe rezultatet e clustering-ut të pambikëqyrur. Të gjitha modelet u testuan mbi të njëjtin pipeline preprocessing, gjë që e bën krahasimin ndërmjet tyre më të drejtë dhe më të besueshëm.

Rezultatet e hyperparameter tuning

Një nga rezultatet kryesore të projektit është tabela e “Best Parameters”, ku për secilin model janë dokumentuar vlerat e testuara dhe konfigurimi final i zgjedhur. Kjo tabelë tregon se procesi i optimizimit nuk është bërë në mënyrë arbitrare, por është mbështetur në eksperimentim sistematik.

Për Logistic Regression, tuning-u i vlerës C përcaktoi nivelin optimal të regularization. Për Decision Tree, parametrat si *max_depth*, *min_samples_split* dhe *min_samples_leaf* ndikuan drejtpërdrejt në kompleksitetin e modelit dhe në aftësinë e tij për të balancuar overfitting-un me generalization. Për KNN, parametrat si numri i fqinjëve, lloji i peshimit dhe metrika e distancës ndihmuan në gjetjen e konfigurimit më të përshtatshëm për strukturën e dataset-it.

Te Neural Network, dy arkitekturat manuale u krahasuan për të parë nëse rritja e thellësisë dhe e numrit të neuroneve jep përfitim të qartë në performancë. Ky krahasim është i vlefshëm sepse dokumenton ndikimin e kompleksitetit të modelit në detyrën e intrusion detection.

Rezultatet e klasifikimit

Performanca e secilit klasifikues u përmbledh në një tabelë me metrikat Accuracy, Precision, Recall dhe F1-score. Kjo tabelë është baza kryesore për krahasimin e modeleve dhe për identifikimin e klasifikuesit më të përshtatshëm për problemin e detektimit binar të intruzioneve.

Në përgjithësi, modelet që kapin marrëdhënie jo-lineare, si Decision Tree ose Neural Network, priren të performojnë më mirë në datasets si NSL-KDD, sepse sjellja e trafikut të rrjetit dhe marrëdhëniet mes veçorive nuk janë domosdoshmërisht lineare. Nga ana tjetër, Logistic Regression ofron një baseline të rëndësishëm dhe KNN paraqet një qasje intuitive, por shpesh ndikohet më shumë nga shkallëzimi dhe madhësia e dataset-it.

Best Parameters Table:		Tested Values	Best Params
Model			
Logistic Regression		C=[0.01, 0.1, 1, 10], penalty='l2', solver='lbfgs'	{'C': 10, 'penalty': 'l2', 'solver': 'lbfgs'}
Decision Tree	max_depth=[5,10,15,None], min_samples_split=[2,5,10], min_samples_leaf=[1,2,4]		{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 5}
KNN	n_neighbors=[3,5,7,9], weights=['uniform','distance'], p=[1,2]		{'n_neighbors': 3, 'p': 1, 'weights': 'distance'}
Neural Network A	layers=[64,32], lr=0.001, dropout=0.3, batch_size=64		Manual architecture experiment
Neural Network B	layers=[128,64,32], lr=0.001, dropout=0.3, batch_size=64		Manual architecture experiment

Tabela 3. Best Parameters Table – testimi i hiperparametrave dhe zgjidhja e konfigurimit më të mirë për secilin model

Final Comparison Table:					
	Model	Accuracy	Precision	Recall	F1
	Decision Tree (Tuned)	0.9983	0.9981	0.9982	0.9982
	KNN (Tuned)	0.9967	0.9968	0.9962	0.9965
	Neural Network A [64, 32]	0.9942	0.9941	0.9935	0.9938
	Neural Network B [128, 64, 32]	0.9939	0.9946	0.9924	0.9935
	Logistic Regression (Tuned)	0.9718	0.9762	0.9631	0.9696

Tabela 4. Final Comparison Table – metrikat e performancës (Accuracy, Precision, Recall, F1-score) për të gjitha tuned models.

```

=== Logistic Regression (Tuned) ===
Accuracy: 97.18%
Precision: 97.62%
Recall: 96.31%
F1-Score: 96.96%
Confusion Matrix:
[[13146  276]
 [ 435 11338]]
-----

=== Decision Tree (Tuned) ===
Accuracy: 99.83%
Precision: 99.81%
Recall: 99.82%
F1-Score: 99.82%
Confusion Matrix:
[[13400  22]
 [ 21 11752]]
-----

=== KNN (Tuned) ===
Accuracy: 99.67%
Precision: 99.68%
Recall: 99.62%
...
Confusion Matrix:
[[13384  38]
 [ 45 11728]]
-----

```

Figura 5. Rezultatet e vlerësimit dhe matricat e konfuzionit për Logistic Regression, Decision Tree dhe KNN.

Confusion matrix për secilin model shton një nivel më të detajuar interpretimi. Kjo lejon identifikimin e:

- Rasteve normale të klasifikuara saktë
- Sulmeve të detektuara saktë
- False positives
- False negatives

Në kontekstin e intrusion detection, false negatives kanë peshë të madhe, sepse përfaqësojnë sulme reale që sistemi nuk i ka kapur. Prandaj, përveç accuracy, u kushtua vëmendje edhe recall-it dhe F1-score-it gjatë interpretimit të rezultateve.

```
=== Neural Network A [64, 32] ===
Accuracy: 99.42%
Precision: 99.41%
Recall: 99.35%
F1-Score: 99.38%
Confusion Matrix:
[[13353   69]
 [   77 11696]]
-----
NN A epochs trained: 19
NN A final train acc: 0.9946044683456421
NN A final val acc: 0.9955348372459412
```

Figura 6. Rezultatet e performancës dhe confusion matrix për Neural Network A

Rezultatet e clustering

Rezultatet e clustering-ut u paraqitën për disa vlera të k dhe u krahasuan me klasat reale përmes ARI dhe NMI. Kjo analizë tregoi sa afër i afrohet ndarja e krijuar nga KMeans strukturës reale të dataset-it.

Kur $k = 2$, rezultati mund të interpretohet si përpjekje për të ndarë të dhënat në dy grupe kryesore që korrespondojnë me klasat normal dhe attack. Nëse ARI dhe NMI dalin më të larta për këtë rast, kjo sugjeron se ndarja binare ka një farë strukture natyrale në të dhëna. Nëse vlerat janë më të larta për $k = 3$ ose $k = 5$, kjo mund të nënkuptojë se klasa “attack” përmban nënstruktura ose nën-grupe që KMeans i identifikon si cluster-a të veçantë.

Vizualizimi me PCA paraqiti të dhënat në dy dimensionë dhe lejoi krahasimin grafik mes cluster-ave të gjeneruar nga KMeans dhe etiketave reale. *explained_variance_ratio* ndihmoi në interpretimin e këtij projeksioni, duke treguar se sa pjesë e variancës totale ruhet nga dy komponentët kryesorë. Nëse dy grafikët duken të ngjashëm, kjo sugjeron një përputhje më të mirë mes clustering-ut dhe labels reale.

5. Diskutimi

Rezultatet e këtij projekti tregojnë se ndërtimi i një sistemi për intrusion detection nuk varet vetëm nga zgjedhja e algoritmit, por edhe nga mënyra se si përgatiten të dhënat, si bëhet tuning-u i parametrave dhe si interpretohen metrikat. Preprocessing-u rezultoi një hap kyç, sepse dataset-i NSL-KDD përmban si veçori kategorike ashtu edhe numerike, dhe pa kodim e shkallëzim të përshtatshëm modelet nuk do të ishin në gjendje të mësonin në mënyrë efikase.

Logistic Regression shërbeu si një baseline i rëndësishëm, sepse ofron një pikë reference të qartë për krahasim. Megjithatë është model linear, ai mund të japë rezultate solide nëse preprocessing-u është i mirë dhe struktura e problemit nuk është shumë komplekse. Megjithatë, për shkak se intrusion detection shpesh përfshin marrëdhënie jo-lineare mes veçorive, modele si Decision Tree ose Neural Network mund të ofrojnë performancë më të lartë.

Decision Tree ofroi avantazhin e kapjes së rregullave jo-lineare dhe të ndërveprimeve komplekse midis veçorive. Nëse ky model ka dalë ndër më të mirët në rezultatet tua, kjo është e kuptueshme, sepse trafiku i rrjetit shpesh përmban modele që nuk mund të përshkruhen lehtë me kufij linearë. Në të njëjtën kohë, parametrat e pemës duhen kontrolluar mirë, sepse pemët shumë të thella mund të mësojnë tepër specifikuish train set-in dhe të humbasin aftësinë për generalization.

KNN paraqiti një qasje intuitive, sepse klasifikimi bëhet duke parë fqinjët më të afërt në hapësirën e veçorive. Megjithatë, kjo qasje është e ndjeshme ndaj scaling-ut dhe bëhet më e kushtueshme kur dataset-i rritet në madhësi. Në projekte si ky, KNN është i dobishëm si model krahasues, por jo gjithmonë është zgjedhja më praktike për datasets me shumë rreshta dhe shumë veçori të koduara.

Dy arkitekturat e Neural Network shtuan një dimension interesant në analizë, sepse treguan se rritja e kompleksitetit nuk sjell domosdoshmërisht përmirësim automatik. Nëse arkitektura më e thellë performon më mirë, kjo tregon se të dhënat përmbajnë marrëdhënie më komplekse; nëse jo, atëherë një model më i thjeshtë mund të jetë më efikas dhe më i lehtë për t'u trajnuar. Kjo thekson rëndësinë e eksperimentimit praktik dhe jo vetëm të supozimit se modeli më i madh është gjithmonë më i mirë.

Në pjesën e clustering-ut, KMeans ofroi një perspektivë plotësuese mbi strukturën e dataset-it. Ndryshe nga klasifikimi i mbikëqyrur, KMeans nuk ka qasje në labels reale dhe bazohet vetëm në ngjashmëritë midis mostrave. Për këtë arsye, është normale që performanca e tij të mos arrijë nivelin e modeleve të mbikëqyrura. Megjithatë, përdorimi i ARI, NMI dhe PCA e bëri këtë pjesë shumë të vlefshme për interpretim, sepse tregoi nëse ekziston një ndarje natyrale e të dhënave që korrespondon me klasat reale.

Një nga kufizimet kryesore të projektit mbetet vetë dataset-i NSL-KDD. Edhe pse është benchmark i përhapur dhe i dobishëm për krahasim metodash, ai nuk përfaqëson plotësisht trafikun modern të rrjeteve reale dhe nuk përfshin të gjitha tiparet e sulmeve bashkëkohore. Kjo do të thotë se rezultatet janë shumë të vlefshme për analizë akademike dhe krahasim algoritmesh, por do të duhej kujdes para se të përgjithësoheshin drejtpërdrejt në një mjedis real prodhimi.

6. Përfundimi

Ky projekt tregoi se machine learning mund të përdoret me sukses për detektimin e intruzioneve në rrjet duke përdorur dataset-in NSL-KDD. Nëpërmjet një pipeline-i të plotë që përfshin preprocessing, trajnimin e klasifikuesve, hyperparameter tuning, evaluim me metrika standarde dhe clustering, u ndërtua një analizë e strukturuar dhe e krahasueshme e metodave kryesore për intrusion detection.

Pjesa e klasifikimit tregoi rëndësinë e zgjedhjes së duhur të modelit dhe të parametrave, ndërsa pjesa e clustering-ut ofroi një këndvështrim shtesë mbi strukturën e brendshme të të dhënave. Përdorimi i PCA për vizualizim dhe i metrikave si ARI dhe NMI e bëri analizën më të plotë dhe më interpretuese.

Në tërësi, projekti përmbush qëllimin kryesor të tij: të ndërtojë dhe vlerësojë disa modele të machine learning për dallimin e trafikut normal nga trafiku sulmues, dhe të tregojë se si qasje të ndryshme mund të krahasohen mbi të njëjtin benchmark. Si zgjerim i ardhshëm, projekti mund të zhvillohet drejt multi-class classification, përdorimit të dataseve më moderne dhe testimit të modeleve më të avancuara ose hibride për intrusion detection.

7. Referencat

1. [NSL-KDD](#) Dataset overview and feature structure.
2. [NSL-KDD](#) record counts, train/test split, and feature description.
3. [NSL-KDD](#) enhanced benchmark description.
4. [PCA](#) explained variance ratio in scikit-learn documentation.
5. [PCA](#) interpretation and variance retention.
6. [Evaluation metrics](#) for intrusion detection models.
7. Supervised classifier comparison on intrusion detection tasks.
8. [Benchmarking datasets for anomaly-based network intrusion detection](#).